

Ch. 12

# 封裝與建構子

## (Encapsulation & Horazon Constructor)

C#程式設計

# 物件導向三大特性

1. **封裝** (Encapsulation)：保護資料，隱藏實作細節。
2. **繼承** (Inheritance)：程式碼共用與擴充。
3. **多型** (Polymorphism)：同一介面，多種實作。

本章將專注於 **封裝**。

# 為什麼要封裝？

在上一章的例子中，我們將變數設為 `public`：

```
class Player {  
    public int hp;  
}  
  
Player p = new Player();  
p.hp = -100; // ⚠ 不合理的數值！
```

直接暴露資料 (Fields) 是危險的，可能導致：

- 資料被設為無效值 (如  $HP < 0$ )。
- 外部程式過度依賴內部實作，導致日後難以修改。

# 存取修飾詞 (Access Modifiers)

C# 透過修飾詞來控制成員的可見度：

- `public`：完全公開，任何人都能存取。
- `private`：**私有**，只有**類別內部**可以存取。(預設值)
- `protected`：只有繼承者可以存取(下章詳談)。

```
class Player {  
    private int hp; // 外部看不到了！  
  
    public void SetHP(int value) {  
        if (value < 0) value = 0; // 保護邏輯  
        hp = value;  
    }  
}
```

# 傳統的 Getter 與 Setter

在沒有屬性語法之前 (如 Java)，我們通常這樣寫：

```
class Player {  
    private int hp;  
  
    // 取得 HP (Getter)  
    public int GetHP() {  
        return hp;  
    }  
  
    // 設定 HP (Setter)  
    public void SetHP(int value) {  
        if (value < 0) value = 0;  
        hp = value;  
    }  
}
```

呼叫稍微麻煩一點：`p.SetHP(100);`

# 屬性 (Property)

C# 提供了一種更優雅的語法來實現封裝，稱為 **屬性**。  
它看起來像變數，但其實是**方法** (Getter & Setter)。

```
class Player {  
    private int _hp; // 私有變數 (backing field)  
  
    public int HP // 公開屬性 (Property)  
    {  
        get { return _hp; }  
        set {  
            if (value < 0) value = 0;  
            _hp = value;  
        }  
    }  
}
```

使用時： `p.HP = -50;` (會自動變為 0)

# 自動實作屬性 (Auto-Implemented Property)

如果不需要特殊的保護邏輯，只是想寫成屬性：

```
class Player {  
    // 編譯器會自動幫你建立一個看不見的 private 變數  
    public string Name { get; set; }  
  
    // 唯讀屬性 (只能在建構子中賦值)  
    public int MaxHP { get; private set; }  
}
```

這是 C# 開發中最常見的寫法！請盡量使用屬性 (Property) 而非公開變數 (Public Field)。

# 建構子 (Constructor)

當我們 `new` 一個物件時，常常需要**初始化**它的狀態。

**建構子** 是一個特殊的方法：

1. **名稱與類別相同**。
2. **沒有回傳型別** (連 `void` 都不寫)。
3. 在 `new` 的瞬間自動執行。

```
class Player {  
    public string Name { get; set; }  
    public int HP { get; set; }  
  
    // 建構子  
    public Player(string name, int hp) {  
        Name = name;  
        HP = hp;  
    }  
}
```



# 使用建構子

```
// 不需要一行一行設定屬性了  
Player p1 = new Player("勇者", 100);  
Player p2 = new Player("魔王", 999);
```

# 物件初始化設定項 (Object\_INITIALIZER)

C# 提供另一種初始化的語法糖，配合屬性使用非常方便：

```
class Item {  
    public string Name { get; set; }  
    public int Price { get; set; }  
}  
  
// 不需要定義建構子，也可以這樣寫：  
Item potion = new Item  
{  
    Name = "紅藥水",  
    Price = 50  
};
```

# 建構子多載 (Constructor Overloading)

如同方法多載，建構子也可以有很多個，只要參數列表不同即可。

```
class Player {  
    public string Name { get; set; }  
    public int HP { get; set; }  
  
    // 1. 無參數建構子 (給預設值)  
    public Player() {  
        Name = "Unknown";  
        HP = 10;  
    }  
  
    // 2. 指定名稱的建構子  
    public Player(string name) {  
        Name = name;  
        HP = 100;  
    }  
  
    // 3. 指定全部의 建構子 (使用 this 呼叫其他建構子)  
    public Player(string name, int hp) : this(name) {  
        HP = hp;  
    }  
}
```

# 解構子 (Destructor)

相對應於建構子，**解構子**是在物件「被回收」前執行的。

寫法是 `~類別名稱()`。

```
class Player {  
    ~Player() {  
        Console.WriteLine("Player 物件被銷毀了...");  
    }  
}
```

**注意：**在 C# (託管語言) 中，記憶體由 **垃圾回收器 (GC)** 自動管理。

我們無法確定解構子什麼時候會執行，因此**非常少用**。除非你需要手動釋放非託管資源 (如檔案串流、C++ DLL)。

# 總結

1. **封裝**：使用 `private` 隱藏細節，使用 `public` 開放介面。
2. **屬性 (Property)**：C# 特有的封裝語法，兼具安全與便利 ( `{ get; set; }` )。
3. **建構子 (Constructor)**：用於物件初始化的特殊方法，名稱與類別相同。

掌握封裝，你的程式碼將更健壯、更安全！



Ch. 12.2

# 靜態 (static)

Horazon

C#程式設計

# 實例 vs. 靜態

到目前為止，我們建立的變數和方法都屬於**物件 (實例)**：

```
Player p1 = new Player("勇者", 100);  
Player p2 = new Player("魔王", 999);  
  
p1.HP = 100; // p1 自己的 HP  
p2.HP = 50;  // p2 自己的 HP，跟 p1 無關
```

每個物件都有**自己的一份**資料。

但有些資料是**全部物件共用的**，例如「目前場上共有幾個玩家？」



# static 關鍵字

在成員前加上 `static`，這個成員就**屬於類別本身**，而不是某個物件。

```
class Player
{
    public string Name { get; set; }
    public int HP { get; set; }

    // 靜態欄位：所有 Player 共用同一份
    public static int Count = 0;

    public Player(string name, int hp)
    {
        Name = name; HP = hp;
        Count++; // 每建立一個 Player，計數 +1
    }
}
```

# 使用靜態成員

靜態成員透過類別名稱存取，不需要建立物件：

```
Player p1 = new Player("勇者", 100);  
Player p2 = new Player("魔王", 999);  
  
// 透過類別名稱存取，不是 p1.Count  
Console.WriteLine(Player.Count); // 2
```

**Math** 類別就是最常見的例子：

```
// 不需要 new Math()，直接用類別名稱呼叫  
double r = Math.Sqrt(16); // 4  
int max = Math.Max(10, 20); // 20
```

# 靜態方法的限制

靜態方法**不能存取**非靜態的成員，因為執行時不知道是哪個物件：

```
class Player
{
    public int HP = 100; // 實例成員

    public static void PrintHP()
    {
        // ✗ 錯誤！HP 屬於哪個物件？不知道！
        Console.WriteLine(HP);
    }
}
```

靜態方法只能操作**靜態成員**或**傳入的參數**。

# 靜態類別 (Static Class)

如果一個類別所有成員都是靜態的，可以直接把類別也設為 `static`：

```
static class GameConfig
{
    public static int MaxPlayers = 4;
    public static float MusicVolume = 0.8f;
    public static string Version = "1.0.0";
}

Console.WriteLine(GameConfig.Version); // 1.0.0
```

靜態類別**不能被** `new`，也不能被繼承。

適合用於設定檔、共用工具方法。

# static 總結

|          | 實例成員      | 靜態成員 (static) |
|----------|-----------|---------------|
| 屬於       | 每個物件自己    | 類別本身          |
| 存取方式     | 物件.成員     | 類別名稱.成員       |
| 需要 new ? | 需要        | 不需要           |
| 適合用途     | 每個物件不同的資料 | 全域共用的資料或工具    |

原則：當資料或行為與「特定物件無關」時，考慮使用 `static`。